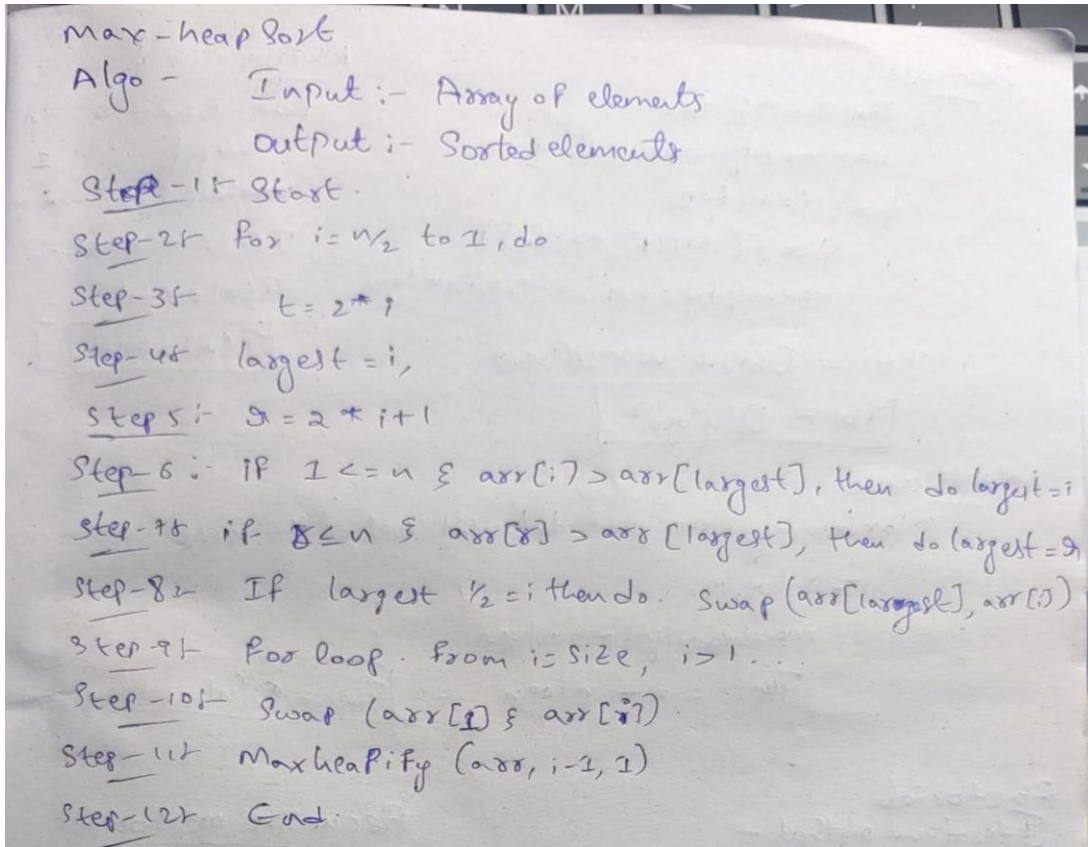


PRACTICAL-03

AIM: Implementation of max-heap sort algorithm.

1) MAX HEAP

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;
void maxheapify(int arr[], int size, int i)
{
    int largest = i;
    int l = 2 * i;
    int r = 2 * i + 1;

    if (l <= size && arr[l] > arr[largest])
    {
        largest = l;
    }

    if (r <= size && arr[r] > arr[largest])
    {
        largest = r;
    }
}
```

```
if (largest != i)
{
    swap(arr[largest], arr[i]);
    maxheapify(arr, size, largest);
}
}

void heapsort(int arr[], int size)
{
    // Build Max Heap
    for (int i = size / 2; i >= 1; i--)
    {
        maxheapify(arr, size, i);
    }

    // Heap Sort
    for (int i = size; i > 1; i--)
    {
        swap(arr[1], arr[i]);
        maxheapify(arr, i - 1, 1);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    ";
    cin >> n;

    int arr[n + 1];

    cout << "Enter elements: ";

    for (int i = 1; i <= n; i++)
    {
        cin >> arr[i];
    }

    heapsort(arr, n);

    cout << "Sorted array: ";

    for (int i = 1; i <= n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

OUTPUT:

```

Output
Enter number of elements: 5
Enter elements: 5
9
7
6
1
Sorted array: 1 5 6 7 9
=== Code Execution Successful ===

```

TIME COMPLEXITY:

Time Complexity

For $(i = n/2, i \geq 1, i--)$ { $\rightarrow n$
 $\text{maxHeapify(arr, n, i);}$ $\rightarrow n \log n$
 For $(i = n, i > 1, i--)$ { $\rightarrow n$
 $\text{Swap(arr[i], arr[1]);}$ $\rightarrow 1$
 $\text{maxHeapify(arr, i-1, 1);}$ $\rightarrow \log n$
 }
 }

$T(n) = n \times \log n + n + 1 \times \log n = n \log n + n \log n$
 $\Rightarrow T(n) = O(n \log n)$

Best, Average, worst case = $O(n \log n)$.

Best case

For $i = n/2$ to 1
 $\text{maxHeapify(arr, n, i);}$ $\rightarrow n$
 For $i = n$ to 2
 $\text{Swap(arr[i], arr[1]);}$
 $\text{maxHeapify(arr, i-1, 1);}$ $\rightarrow n \log n$
 • $T(n) = O(n) + n \times O(\log n)$
 $T(n) = O(n \log n)$

Worst case

Max heapify from root to lowest level of the heap.
 $\Rightarrow \log_2 n$
 maxHeapify(1); $\rightarrow n \log n$
 $T(n) = n + n \log n \Rightarrow T(n) = O(n \log n)$

Average case

For $i = n$ to 2 $\rightarrow n$
 Swap() $\rightarrow 1$
 maxHeapify() $\rightarrow \log n$
 $T(n) = n + n \log n$
 $T(n) = n + n \log n$
 $T(n) = O(n \log n)$

Conclusion: Thus, the Max Heap Sort algorithm was successfully implemented. It efficiently sorts elements using a max heap with a time complexity of $O(n \log n)$.